



Reinforcement Learning

P Satya Jayadev

Web Enabled M.Tech,
Indian Institute of Technology Madras

Topic 5

Policy Gradient Methods



1. Intro to Policy Gradient Methods
2. Policy Parameterisation
3. Policy Optimization
4. Policy Gradient Theorem
5. REINFORCE algorithm

References: Sutton, R. S., Barto, A. G. (2018). Reinforcement Learning: An Introduction. The MIT Press.



Why Learn the Policy Directly?

Value-function based approach

- ▶ Learn how good each action is in a state: $q(s, a)$
- ▶ Select the action with the highest estimated value: $a^*(s) = \arg \max_a q(s, a)$



Why Learn the Policy Directly?

Value-function based approach

- ▶ Learn how good each action is in a state: $q(s, a)$
- ▶ Select the action with the highest estimated value: $a^*(s) = \arg \max_a q(s, a)$

Where can this become difficult?

- ▶ In a **continuous action space**, finding the action that maximises $q(s, a)$ may require solving a separate optimisation problem
- ▶ Small changes in estimated action values can cause an abrupt change in the selected action.
- ▶ A greedy action-value policy is not suitable where a **stochastic policy** is preferable.



Why Learn the Policy Directly?

Value-function based approach

- ▶ Learn how good each action is in a state: $q(s, a)$
- ▶ Select the action with the highest estimated value: $a^*(s) = \arg \max_a q(s, a)$

Where can this become difficult?

- ▶ In a **continuous action space**, finding the action that maximises $q(s, a)$ may require solving a separate optimisation problem
- ▶ Small changes in estimated action values can cause an abrupt change in the selected action.
- ▶ A greedy action-value policy is not suitable where a **stochastic policy** is preferable.

Key Question

Can the agent learn the policy directly, without first learning the value of every possible action?



Why Learn the Policy Directly?

Value-function based approach

- ▶ Learn how good each action is in a state: $q(s, a)$
- ▶ Select the action with the highest estimated value: $a^*(s) = \arg \max_a q(s, a)$

Where can this become difficult?

- ▶ In a **continuous action space**, finding the action that maximises $q(s, a)$ may require solving a separate optimisation problem
- ▶ Small changes in estimated action values can cause an abrupt change in the selected action.
- ▶ A greedy action-value policy is not suitable where a **stochastic policy** is preferable.

Key Question

Can the agent learn the policy directly, without first learning the value of every possible action?

Yes. **Policy-gradient methods** directly adjust the policy parameters to increase the expected long-term return.



Policy Gradient Methods

Where Are Policy Gradients Useful?

Policy gradients are particularly useful when actions are **continuous**, the action space is **very large**, or the desired behaviour should remain **stochastic**.

Some Potential Use Cases:

Robotic Control

State: Camera images, joint positions and forces

Action: Joint movement or torque

Reward: Task success, smooth movement and safety

Industrial Control

State: Temperature, pressure, quality and energy use

Action: Continuous adjustment of a valve or set point

Reward: Product quality minus energy and violation costs

LLM Tuning

State: Prompt and previously generated tokens **Action:**

Probability distribution over the next token **Reward:** Score representing preferred model behaviour



Where Are Policy Gradients Useful?

Policy gradients are particularly useful when actions are **continuous**, the action space is **very large**, or the desired behaviour should remain **stochastic**.

Some Potential Use Cases:

Robotic Control

State: Camera images, joint positions and forces

Action: Joint movement or torque

Reward: Task success, smooth movement and safety

Industrial Control

State: Temperature, pressure, quality and energy use

Action: Continuous adjustment of a valve or set point

Reward: Product quality minus energy and violation costs

LLM Tuning

State: Prompt and previously generated tokens **Action:**

Probability distribution over the next token **Reward:** Score representing preferred model behaviour

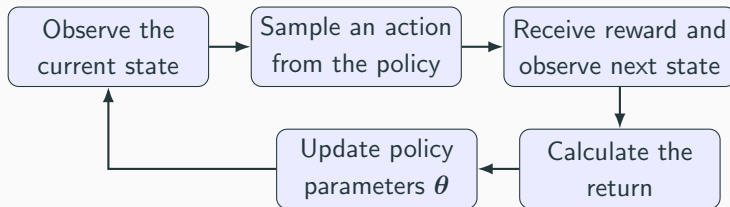
Common Idea

Instead of evaluating every possible action separately, learn a policy that directly produces an appropriate action distribution for the current state.



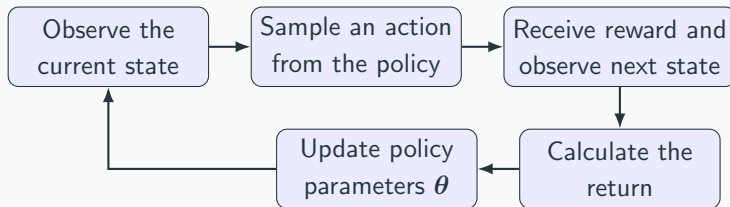
How Does Policy-Gradient Learning Work?

A policy $\pi_{\theta}(a|s)$ uses parameters θ to determine the probability of selecting each action.



How Does Policy-Gradient Learning Work?

A policy $\pi_{\theta}(a|s)$ uses parameters θ to determine the probability of selecting each action.



When the return is high

Increase the probability of actions that contributed to the outcome.

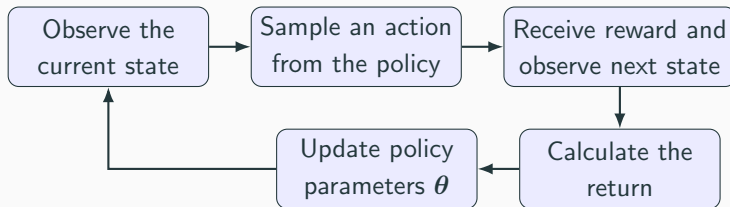
When the return is low

Reduce the probability of actions that contributed to the outcome.



How Does Policy-Gradient Learning Work?

A policy $\pi_{\theta}(a|s)$ uses parameters θ to determine the probability of selecting each action.



When the return is high

Increase the probability of actions that contributed to the outcome.

Central Idea

Policy-gradient methods improve behaviour by changing action probabilities in the direction of higher expected return.

When the return is low

Reduce the probability of actions that contributed to the outcome.



Policy-Gradient Methods

Policy-gradient methods learn the policy directly.

- The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$



Policy-Gradient Methods

Policy-gradient methods learn the policy directly.

- ▶ The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$
- ▶ For a **discrete action**, $\pi_{\theta}(a|s)$ gives the probability of selecting action a in state s .



Policy-Gradient Methods

Policy-gradient methods learn the policy directly.

- ▶ The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$
- ▶ For a **discrete action**, $\pi_{\theta}(a|s)$ gives the probability of selecting action a in state s .
- ▶ For a **continuous action**, it describes a probability density over possible action values.



Policy-Gradient Methods

Policy-gradient methods learn the policy directly.

- ▶ The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$
- ▶ For a **discrete action**, $\pi_{\theta}(a|s)$ gives the probability of selecting action a in state s .
- ▶ For a **continuous action**, it describes a probability density over possible action values.
- ▶ Learning adjusts θ to maximise the expected long-term return:

$$\theta^* = \arg \max_{\theta} J(\theta),$$



Policy-gradient methods learn the policy directly.

- ▶ The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$
- ▶ For a **discrete action**, $\pi_{\theta}(a|s)$ gives the probability of selecting action a in state s .
- ▶ For a **continuous action**, it describes a probability density over possible action values.
- ▶ Learning adjusts θ to maximise the expected long-term return:

$$\theta^* = \arg \max_{\theta} J(\theta),$$

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)]$$



Policy-gradient methods learn the policy directly.

- ▶ The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$
- ▶ For a **discrete action**, $\pi_{\theta}(a|s)$ gives the probability of selecting action a in state s .
- ▶ For a **continuous action**, it describes a probability density over possible action values.
- ▶ Learning adjusts θ to maximise the expected long-term return:

$$\theta^* = \arg \max_{\theta} J(\theta),$$

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)]$$

Why Use a Stochastic Policy?

A stochastic policy supports exploration and can represent situations where more than one action may be appropriate in the same state.



Policy-gradient methods learn the policy directly.

- ▶ The policy is represented by a differentiable function with parameters θ : $\pi_{\theta}(a|s)$
- ▶ For a **discrete action**, $\pi_{\theta}(a|s)$ gives the probability of selecting action a in state s .
- ▶ For a **continuous action**, it describes a probability density over possible action values.
- ▶ Learning adjusts θ to maximise the expected long-term return:

$$\theta^* = \arg \max_{\theta} J(\theta),$$

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)]$$

Why Use a Stochastic Policy?

A stochastic policy supports exploration and can represent situations where more than one action may be appropriate in the same state.

Scope of This Topic

We focus on stochastic policy gradients. Deterministic policy-gradient methods will be introduced later through DDPG.



Policy Parameterisation for Discrete Actions

- ▶ For a small or moderately sized discrete action space, the policy network produces a numerical **preference** for every action: $h_{\theta}(s, a) \in \mathbb{R}$
- ▶ A higher preference indicates that the action should be selected more often in state s .



Policy Parameterisation for Discrete Actions

- ▶ For a small or moderately sized discrete action space, the policy network produces a numerical **preference** for every action: $h_{\theta}(s, a) \in \mathbb{R}$
- ▶ A higher preference indicates that the action should be selected more often in state s .
- ▶ The softmax function converts these preferences into probabilities:

$$\pi_{\theta}(a|s) = \frac{\exp(h_{\theta}(s, a))}{\sum_{a' \in \mathcal{A}(s)} \exp(h_{\theta}(s, a'))}$$

- ▶ Advantage of softmax function is the policy can be stochastic and also approach a deterministic policy if needed



Policy Parameterisation for Discrete Actions

- ▶ For a small or moderately sized discrete action space, the policy network produces a numerical **preference** for every action: $h_{\theta}(s, a) \in \mathbb{R}$
- ▶ A higher preference indicates that the action should be selected more often in state s .
- ▶ The softmax function converts these preferences into probabilities:

$$\pi_{\theta}(a|s) = \frac{\exp(h_{\theta}(s, a))}{\sum_{a' \in \mathcal{A}(s)} \exp(h_{\theta}(s, a'))}$$

- ▶ Advantage of softmax function is the policy can be stochastic and also approach a deterministic policy if needed

Example: Policy output for a speed control action

Action	Preference	Probability
Reduce speed	0	0.245
Maintain speed	1	0.665
Increase speed	-1	0.090



Policy Parameterisation for Continuous Actions

- ▶ When actions can take any value within a range, assigning a separate probability to every action is not possible.



Policy Parameterisation for Continuous Actions

- ▶ When actions can take any value within a range, assigning a separate probability to every action is not possible.
- ▶ The policy can instead produce a probability distribution over actions.

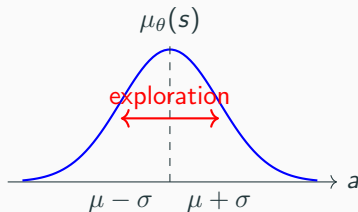


Policy Parameterisation for Continuous Actions

- ▶ When actions can take any value within a range, assigning a separate probability to every action is not possible.
- ▶ The policy can instead produce a probability distribution over actions.
- ▶ A common choice is a Gaussian policy:

$$\pi_{\theta}(a|s) = \frac{1}{\sigma_{\theta}(s)\sqrt{2\pi}} \exp \left[-\frac{(a - \mu_{\theta}(s))^2}{2\sigma_{\theta}^2(s)} \right]$$

- ▶ $\mu_{\theta}(s)$: action currently preferred by the policy.
- ▶ $\sigma_{\theta}(s)$: amount of exploration around that action.



Example: Mountain Car with Continuous Actions

► Problem Statement

- Objective: Reach the top of the right hill, $x_t \geq 0.45$, while limiting control effort.
- State Vector: $s_t = [\text{position}, \text{velocity}]$
- Continuous action: $a_t \in [-1, +1]$, where a_t represents the applied engine force.
- The episode ends when the goal is reached or after 999 steps. Each transition represents 0.02 seconds.



Example: Mountain Car with Continuous Actions

► Problem Statement

- Objective: Reach the top of the right hill, $x_t \geq 0.45$, while limiting control effort.
- State Vector: $s_t = [\text{position}, \text{velocity}]$
- Continuous action: $a_t \in [-1, +1]$, where a_t represents the applied engine force.
- The episode ends when the goal is reached or after 999 steps. Each transition represents 0.02 seconds.



[Documentation Link](#)

- **Gaussian Policy** For the current state, the policy produces a preferred action $\mu_\theta(s_t)$ and an exploration level $\sigma_\theta(s_t)$:

$$\tilde{a}_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma_\theta^2(s_t)), \quad a_t = \text{clip}(\tilde{a}_t, -1, +1)$$

- $\mu_\theta(s_t)$ controls the preferred direction and strength of the force.
- $\sigma_\theta(s_t)$ controls how widely the policy explores nearby actions.



Example: Mountain Car with Continuous Actions

► Reward Function

$$r_t = \begin{cases} 100 - 0.1a_t^2, & \text{if the car reaches } x_t \geq 0.45, \\ -0.1a_t^2, & \text{otherwise.} \end{cases}$$

The agent receives a success bonus of +100, while the term $-0.1a_t^2$ discourages excessive use of engine force.



Example: Mountain Car with Continuous Actions

► Reward Function

$$r_t = \begin{cases} 100 - 0.1a_t^2, & \text{if the car reaches } x_t \geq 0.45, \\ -0.1a_t^2, & \text{otherwise.} \end{cases}$$

The agent receives a success bonus of +100, while the term $-0.1a_t^2$ discourages excessive use of engine force.

- **Learning Challenge** The car must initially move away from the goal to gain momentum. Hence, actions that appear immediately unhelpful may produce a higher long-term return.



[Documentation Link](#)



Return from the Trajectory

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t R_{t+1}$$

A trajectory with a larger $G(\tau)$ represents a better outcome for the agent.



Return from the Trajectory

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t R_{t+1}$$

A trajectory with a larger $G(\tau)$ represents a better outcome for the agent.

Probability of Generating the Trajectory

Under policy π_θ , the probability of trajectory τ is:

$$D^{\pi_\theta}(\tau) = p(S_0) \prod_{t=0}^{T-1} \pi_\theta(A_t|S_t) p(S_{t+1}|S_t, A_t)$$

Connection to Policy Learning

Changing θ changes the action probabilities and therefore changes which trajectories are likely to occur.



Objective of Policy Optimisation

A stochastic policy can generate many different trajectories. The quality of the policy is measured by its **expected return**:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$



Objective of Policy Optimisation

A stochastic policy can generate many different trajectories. The quality of the policy is measured by its **expected return**:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$

Optimisation Objective

Find the policy parameters that produce the highest expected return:

$$\theta^* = \arg \max_{\theta} J(\theta)$$



Objective of Policy Optimisation

A stochastic policy can generate many different trajectories. The quality of the policy is measured by its **expected return**:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$

Optimisation Objective

Find the policy parameters that produce the highest expected return:

$$\theta^* = \arg \max_{\theta} J(\theta)$$

Intuition

Policy optimisation should make high-return trajectories more likely and low-return trajectories such as less likely.



Why Is Policy Optimisation Difficult?

The expected return depends on the probability of complete trajectories:

$$J(\theta) = \sum_{\tau} D^{\pi_{\theta}}(\tau) G(\tau)$$
$$D^{\pi_{\theta}}(\tau) = p(S_0) \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(A_t|S_t)}_{\text{policy}} \underbrace{p(S_{t+1}|S_t, A_t)}_{\text{environment dynamics}}$$

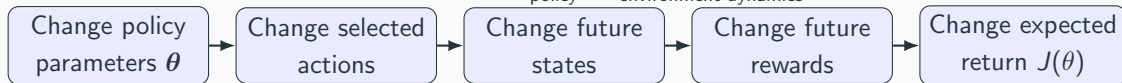


Why Is Policy Optimisation Difficult?

The expected return depends on the probability of complete trajectories:

$$J(\theta) = \sum_{\tau} D^{\pi_{\theta}}(\tau) G(\tau)$$

$$D^{\pi_{\theta}}(\tau) = p(S_0) \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(A_t|S_t)}_{\text{policy}} \underbrace{p(S_{t+1}|S_t, A_t)}_{\text{environment dynamics}}$$

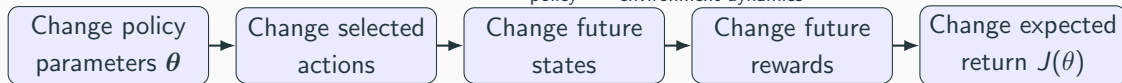


Why Is Policy Optimisation Difficult?

The expected return depends on the probability of complete trajectories:

$$J(\theta) = \sum_{\tau} D^{\pi_{\theta}}(\tau) G(\tau)$$

$$D^{\pi_{\theta}}(\tau) = p(S_0) \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(A_t|S_t)}_{\text{policy}} \underbrace{p(S_{t+1}|S_t, A_t)}_{\text{environment dynamics}}$$



The Apparent Problem

To compute how a change in θ affects the final return, a naive end-to-end approach would differentiate through every subsequent state transition:

$$\theta \longrightarrow A_t \longrightarrow S_{t+1} \longrightarrow A_{t+1} \longrightarrow \cdots \longrightarrow G(\tau)$$

This would require a known and differentiable model of $p(S_{t+1}|S_t, A_t)$ which is not possible.



Policy Gradient: From Expected Return to a Gradient

The objective is to maximise the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$



Policy Gradient: From Expected Return to a Gradient

The objective is to maximise the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$

Differentiating with respect to the policy parameters:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} D^{\pi_\theta}(\tau) G(\tau)$$



Policy Gradient: From Expected Return to a Gradient

The objective is to maximise the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$

Differentiating with respect to the policy parameters:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} D^{\pi_\theta}(\tau) G(\tau)$$

Using the log-derivative identity

$$\nabla_{\theta} D = D \nabla_{\theta} \log D,$$

we obtain:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau) \nabla_{\theta} \log D^{\pi_\theta}(\tau)]$$



Policy Gradient: From Expected Return to a Gradient

The objective is to maximise the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau)] = \sum_{\tau} D^{\pi_\theta}(\tau) G(\tau)$$

Differentiating with respect to the policy parameters:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} D^{\pi_\theta}(\tau) G(\tau)$$

Using the log-derivative identity

$$\nabla_{\theta} D = D \nabla_{\theta} \log D,$$

we obtain:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_\theta}} [G(\tau) \nabla_{\theta} \log D^{\pi_\theta}(\tau)]$$

Interpretation

- ▶ $\nabla_{\theta} \log D^{\pi_\theta}(\tau)$ gives the direction that makes trajectory τ more likely.
- ▶ $G(\tau)$ determines how strongly that trajectory should influence the policy update.



Policy Gradient Theorem

The probability of a trajectory contains both policy decisions and environment transitions:

$$\log D^{\pi_{\theta}}(\tau) = \log p(S_0) + \sum_{t=0}^{T-1} \left[\underbrace{\log \pi_{\theta}(A_t | S_t)}_{\text{depends on } \theta} + \underbrace{\log p(S_{t+1} | S_t, A_t)}_{\text{does not depend on } \theta} \right]$$



Policy Gradient Theorem

The probability of a trajectory contains both policy decisions and environment transitions:

$$\log D^{\pi_{\theta}}(\tau) = \log p(S_0) + \sum_{t=0}^{T-1} \left[\underbrace{\log \pi_{\theta}(A_t|S_t)}_{\text{depends on } \theta} + \underbrace{\log p(S_{t+1}|S_t, A_t)}_{\text{does not depend on } \theta} \right]$$

Therefore, only the policy terms remain after differentiation:

$$\nabla_{\theta} \log D^{\pi_{\theta}}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)$$



Policy Gradient Theorem

The probability of a trajectory contains both policy decisions and environment transitions:

$$\log D^{\pi_{\theta}}(\tau) = \log p(S_0) + \sum_{t=0}^{T-1} \left[\underbrace{\log \pi_{\theta}(A_t|S_t)}_{\text{depends on } \theta} + \underbrace{\log p(S_{t+1}|S_t, A_t)}_{\text{does not depend on } \theta} \right]$$

Therefore, only the policy terms remain after differentiation:

$$\nabla_{\theta} \log D^{\pi_{\theta}}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)$$

Substituting this into the previous result gives:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim D^{\pi_{\theta}}} \left[G(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t|S_t) \right]$$



Policy Gradient Theorem

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau} \left[G(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \\&= \sum_{t=0}^{T-1} \mathbb{E}_{\tau} [G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] \quad (\text{linearity of expectation}) \\&= \sum_{t=0}^{T-1} \gamma^t \mathbb{E}_{\tau} [G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] \quad (\text{causality}) \\&= \sum_{t=0}^{T-1} \gamma^t \mathbb{E}_{S_t, A_t} [q^{\pi_{\theta}}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)].\end{aligned}$$

- ▶ An action at time t cannot influence rewards received before t ; hence $G(\tau)$ is replaced by $\gamma^t G_t$.
- ▶ Conditioning on S_t and A_t replaces the sampled return G_t by its expectation $q^{\pi_{\theta}}(S_t, A_t)$.



Policy Gradient Theorem

Define the discounted state-visitation weighing as:

$$d^{\pi_{\theta}}(s) = \sum_{t=0}^{T-1} \gamma^t \Pr(S_t = s \mid \pi_{\theta}).$$

- ▶ It measures how frequently the agent is expected to visit state s while following the current policy.
- ▶ The factor γ^t gives less weight to visits occurring later; therefore, states visited frequently or earlier have a greater influence on the policy update.



Policy Gradient Theorem

Define the discounted state-visitation weighing as:

$$d^{\pi_{\theta}}(s) = \sum_{t=0}^{T-1} \gamma^t \Pr(S_t = s \mid \pi_{\theta}).$$

- ▶ It measures how frequently the agent is expected to visit state s while following the current policy.
- ▶ The factor γ^t gives less weight to visits occurring later; therefore, states visited frequently or earlier have a greater influence on the policy update.

Therefore:

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{\substack{S \sim d^{\pi_{\theta}} \\ A \sim \pi_{\theta}(\cdot|S)}} [q^{\pi_{\theta}}(S, A) \nabla_{\theta} \log \pi_{\theta}(A|S)]$$



Policy Gradient Theorem

Define the discounted state-visitation weighing as:

$$d^{\pi_{\theta}}(s) = \sum_{t=0}^{T-1} \gamma^t \Pr(S_t = s \mid \pi_{\theta}).$$

- ▶ It measures how frequently the agent is expected to visit state s while following the current policy.
- ▶ The factor γ^t gives less weight to visits occurring later; therefore, states visited frequently or earlier have a greater influence on the policy update.

Therefore:

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{\substack{S \sim d^{\pi_{\theta}} \\ A \sim \pi_{\theta}(\cdot|S)}} [q^{\pi_{\theta}}(S, A) \nabla_{\theta} \log \pi_{\theta}(A|S)]$$

What Policy Gradients Achieve

The environment determines which states and rewards are observed, but we do not differentiate through it. We require only sampled trajectories and the gradient of the policy.



REINFORCE: Monte Carlo Policy Gradient

- ▶ The Policy Gradient Theorem contains the unknown q-value: $q^{\pi_{\theta}}(S_t, A_t)$
- ▶ REINFORCE replaces it with the sampled return: $q^{\pi_{\theta}}(S_t, A_t) \approx G_t$



REINFORCE: Monte Carlo Policy Gradient

- ▶ The Policy Gradient Theorem contains the unknown q-value: $q^{\pi_\theta}(S_t, A_t)$
- ▶ REINFORCE replaces it with the sampled return: $q^{\pi_\theta}(S_t, A_t) \approx G_t$

REINFORCE Algorithm

Input: Differentiable policy $\pi_\theta(a|s)$, learning rate $\alpha > 0$, and discount factor γ .

1. Generate a complete episode using the current policy π_θ .
2. For each time step $t = 0, 1, \dots, T - 1$:
 - 2.1 Compute the return-to-go G_t .
 - 2.2 Estimate the policy gradient: $\widehat{\nabla_\theta J(\theta)} = \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$
 - 2.3 Perform gradient ascent: $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$



REINFORCE: Monte Carlo Policy Gradient

- ▶ The Policy Gradient Theorem contains the unknown q-value: $q^{\pi_\theta}(S_t, A_t)$
- ▶ REINFORCE replaces it with the sampled return: $q^{\pi_\theta}(S_t, A_t) \approx G_t$

REINFORCE Algorithm

Input: Differentiable policy $\pi_\theta(a|s)$, learning rate $\alpha > 0$, and discount factor γ .

1. Generate a complete episode using the current policy π_θ .
2. For each time step $t = 0, 1, \dots, T - 1$:
 - 2.1 Compute the return-to-go G_t .
 - 2.2 Estimate the policy gradient: $\widehat{\nabla_\theta J(\theta)} = \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$
 - 2.3 Perform gradient ascent: $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$

Key Idea

Actions followed by larger returns are made more likely. REINFORCE learns directly from complete episodes without an environment model or a separately learned value function.



Recall Mountain Car Example: Producing a Continuous Action

The policy receives the current state:

$$s_t = [\text{position}_t, \text{velocity}_t]$$



Recall Mountain Car Example: Producing a Continuous Action

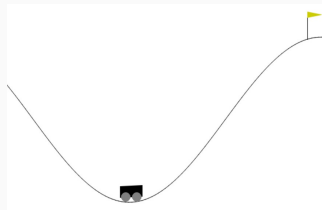
The policy receives the current state:

$$s_t = [\text{position}_t, \text{velocity}_t]$$

The policy network produces two outputs: $\mu_\theta(s_t)$ and $\sigma_\theta(s_t)$. An engine force is sampled from the Gaussian policy:

$$\tilde{a}_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma_\theta^2(s_t))$$

The action is restricted to the valid range: $a_t = \text{clip}(\tilde{a}_t, -1, +1)$



Recall Mountain Car Example: Producing a Continuous Action

The policy receives the current state:

$$s_t = [\text{position}_t, \text{velocity}_t]$$

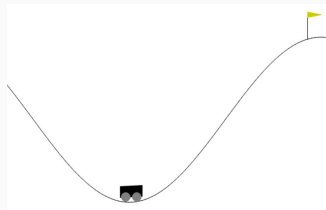
The policy network produces two outputs: $\mu_\theta(s_t)$ and $\sigma_\theta(s_t)$. An engine force is sampled from the Gaussian policy:

$$\tilde{a}_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma_\theta^2(s_t))$$

The action is restricted to the valid range: $a_t = \text{clip}(\tilde{a}_t, -1, +1)$

Meaning of the Policy Outputs

- ▶ $\mu_\theta(s_t)$: preferred direction and strength of force.
- ▶ $\sigma_\theta(s_t)$: amount of exploration around that force.



Mountain Car: Learning with REINFORCE

Learning Process

1. Generate a complete episode using the current Gaussian policy.



Mountain Car: Learning with REINFORCE

Learning Process

1. Generate a complete episode using the current Gaussian policy.
2. Calculate the return G_t for every action taken.



Mountain Car: Learning with REINFORCE

Learning Process

1. Generate a complete episode using the current Gaussian policy.
2. Calculate the return G_t for every action taken.
3. Update the policy parameters using:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



Mountain Car: Learning with REINFORCE

Learning Process

1. Generate a complete episode using the current Gaussian policy.
2. Calculate the return G_t for every action taken.
3. Update the policy parameters using:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Behaviour Learned

The policy gradually learns to:

- ▶ move left initially to gain momentum;
- ▶ reverse direction at an appropriate position;
- ▶ accelerate right to reach the goal;
- ▶ avoid unnecessarily large forces.



Mountain Car: Learning with REINFORCE

Learning Process

1. Generate a complete episode using the current Gaussian policy.
2. Calculate the return G_t for every action taken.
3. Update the policy parameters using:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Key Idea

If an action taken at a particular position and velocity leads to a high return, REINFORCE increases the probability of selecting a similar action in similar states. Actions associated with poor returns become less likely.

Behaviour Learned

The policy gradually learns to:

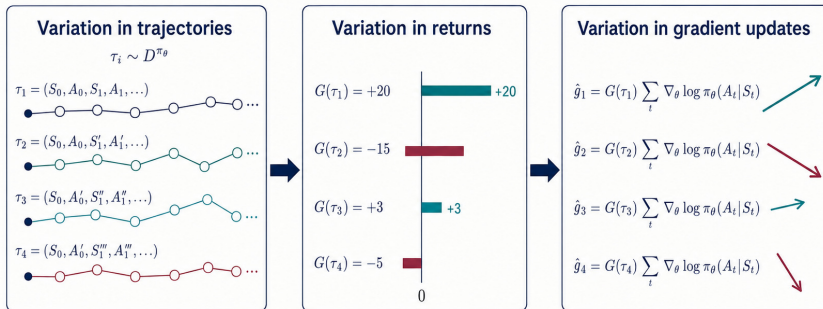
- ▶ move left initially to gain momentum;
- ▶ reverse direction at an appropriate position;
- ▶ accelerate right to reach the goal;
- ▶ avoid unnecessarily large forces.



Limitations of Reinforce

High variance:

- ▶ Gradient is estimated based on sample trajectories returns
- ▶ As the size of episodes increase, returns can differ greatly across sampled episodes, producing noisy and unstable gradient estimates.



Limitations of Reinforce

- ▶ **High variance:**
 - Gradient is estimated based on sample trajectories returns
 - As the size of episodes increase, returns can differ greatly across sampled episodes, producing noisy and unstable gradient estimates.
- ▶ **Low Sample Efficiency:** REINFORCE is on-policy; experience generated by an older policy cannot be reused freely after the policy changes.



Limitations of Reinforce

- ▶ **High variance:**
 - Gradient is estimated based on sample trajectories returns
 - As the size of episodes increase, returns can differ greatly across sampled episodes, producing noisy and unstable gradient estimates.
- ▶ **Low Sample Efficiency:** REINFORCE is on-policy; experience generated by an older policy cannot be reused freely after the policy changes.
- ▶ **Slow Convergence:** Since it is an on-policy learning and gradients could be noisy, updates should be done cautiously with small value of α .

Note

These limitations motivate variance-reduction methods and actor-critic approaches, which learn a value function to provide a more informative policy-update signal.



Reducing Variance with a Baseline

To reduce the variance in the gradient update of REINFORCE:, a baseline $b(S_t)$ can be subtracted from the return:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$



Reducing Variance with a Baseline

To reduce the variance in the gradient update of REINFORCE:, a baseline $b(S_t)$ can be subtracted from the return:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

A common baseline is the estimated state value:

$$b(S_t) = \hat{V}(S_t)$$

Result: It is proven that this baseline reduces variance by most without introducing any bias.



Summary of Policy-Gradient Methods

- Policy-gradient methods learn a parameterised policy $\pi_{\theta}(a|s)$ directly instead of deriving it from action values.



Summary of Policy-Gradient Methods

- ▶ Policy-gradient methods learn a parameterised policy $\pi_{\theta}(a|s)$ directly instead of deriving it from action values.
- ▶ The policy gradient increases the probability of actions associated with higher expected returns



Summary of Policy-Gradient Methods

- ▶ Policy-gradient methods learn a parameterised policy $\pi_{\theta}(a|s)$ directly instead of deriving it from action values.
- ▶ The policy gradient increases the probability of actions associated with higher expected returns
- ▶ The gradient can be estimated from sampled trajectories without differentiating the environment dynamics.



Summary of Policy-Gradient Methods

- ▶ Policy-gradient methods learn a parameterised policy $\pi_{\theta}(a|s)$ directly instead of deriving it from action values.
- ▶ The policy gradient increases the probability of actions associated with higher expected returns
- ▶ The gradient can be estimated from sampled trajectories without differentiating the environment dynamics.
- ▶ REINFORCE uses the observed return G_t as a Monte Carlo estimate of the action value.



Summary of Policy-Gradient Methods

- ▶ Policy-gradient methods learn a parameterised policy $\pi_{\theta}(a|s)$ directly instead of deriving it from action values.
- ▶ The policy gradient increases the probability of actions associated with higher expected returns
- ▶ The gradient can be estimated from sampled trajectories without differentiating the environment dynamics.
- ▶ REINFORCE uses the observed return G_t as a Monte Carlo estimate of the action value.
- ▶ REINFORCE remains on-policy and suffers from high variance, low sample efficiency and noisy updates.



Summary of Policy-Gradient Methods

- ▶ Policy-gradient methods learn a parameterised policy $\pi_{\theta}(a|s)$ directly instead of deriving it from action values.
- ▶ The policy gradient increases the probability of actions associated with higher expected returns
- ▶ The gradient can be estimated from sampled trajectories without differentiating the environment dynamics.
- ▶ REINFORCE uses the observed return G_t as a Monte Carlo estimate of the action value.
- ▶ REINFORCE remains on-policy and suffers from high variance, low sample efficiency and noisy updates.
- ▶ Subtracting a value baseline from the return reduces the variance of the update.

